

Analisi e Progetto di Algoritmi - Teoria

0. Ripasso: notazione theta, O-grande, omega-grande	2
1. Problemi risolubili con la programmazione dinamica	3
2. Weighted Interval Scheduling	4
3. Knapsack 0/1 vs Knapsack frazionario. Differenze concettuali e nell'approccio tramite programmazione	5
4. LCS	6
5. LGCS	7
6. Partition	8
7. Subset Sum	9
8. Schema generale algoritmo greedy e tempo di calcolo	10
9. Cos'è un sistema di indipendenza? Cos'è un matroide? Cosa afferma il teorema di Rado?	11
Prima di proseguire: un riassunto sui grafi	12
10. Rappresentazione in memoria di un grafo	13
11. Algoritmo BFS	14
12. Algoritmo DFS	17
13. Algoritmo di Dijkstra	21
14. Bellman-Ford	24
15. Algoritmi per la costruzione di alberi di copertura minimi: Kruskal e Prim	25
16. Floyd-Warshall	29
17. Strutture dati per insiemi disgiunti	31
18. Reti di flusso e Ford-Fulkerson	34
19. NP completezza	38

o. Ripasso: notazione theta, O-grande, omega-grande

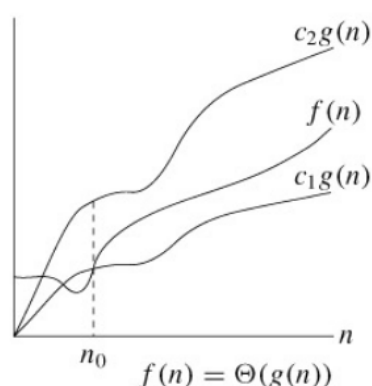
$T : \mathbb{N} \rightarrow \mathbb{N} \quad \forall n \in \mathbb{N}$. n rappresenta la dimensione dell'input di un problema.

$T(n)$ è il costo computazionale, ovvero quante istruzioni l'algoritmo esegue quando ha in input qualcosa la cui dimensione è n .

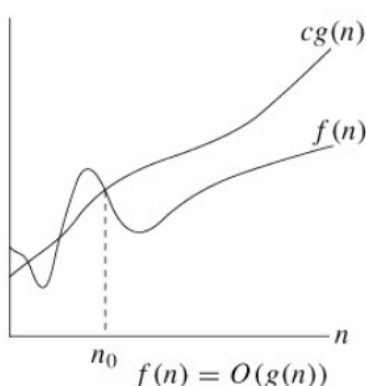
$$\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2 \in \mathbb{R} \wedge n_0 \in \mathbb{N} : \forall n \geq n_0 \quad c_1 g(n) \leq f(n) \leq c_2 g(n)\}$$

$$O(g(n)) = \{f(n) \mid \exists c \in \mathbb{R} \wedge n_0 \in \mathbb{N} : \forall n \geq n_0 \quad f(n) \leq c \cdot g(n)\}$$

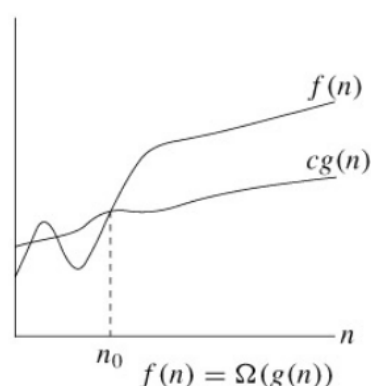
$$\Omega(g(n)) = \{f(n) \mid \exists c \in \mathbb{R} \wedge n_0 \in \mathbb{N} : \forall n \geq n_0 \quad c \cdot g(n) \leq f(n)\}$$



(a)



(b)



(c)

$$3n^2 = \Theta(n^2) \quad 3n^2 = \Omega(n^2) \quad 3n^2 = O(n^2) \quad 4n^2 - 2n = \Theta(n^2)$$

$$P_k(n) = \Theta(n^k) \text{ (polinomio di grado } k) \quad \Theta(1) = \text{costante}$$

$$\log(n) = \Theta(\log(n)) \quad \log(n) = O(n) \quad \log(n) = \Omega(1)$$

1. Problemi risolubili con la programmazione dinamica

I problemi risolubili con la programmazione dinamica sono problemi di ottimizzazione in cui **da un insieme** (generalmente molto **grande**) di **soluzioni possibili** vogliamo estrarre **una** tra le **soluzioni ottime** rispetto ad una particolare misura.

Per poter applicare la programmazione dinamica occorre:

1. Che una soluzione ottima si possa esprimere in termini di soluzioni ottime di sottoproblemi:

proprietà della sottostruttura ottima

2. che l'**insieme dei sottoproblemi distinti** abbia **cardinalità molto inferiore all'insieme delle soluzioni possibili** da cui vogliamo selezionare quella ottima. Se ciò è vero significa che **uno stesso problema deve comparire molte volte come sottoproblema di altri problemi**:

proprietà di sovrapposizione dei sottoproblemi

Un algoritmo di Programmazione Dinamica viene utilizzato per risolvere un problema altrimenti ricorsivo.

La logica è: parto dai problemi più piccoli e, man mano, mi muovo verso la soluzione del problema posto, salvando le soluzioni dei problemi più piccoli.

Questo consente di avere una maggiore efficacia in termini di tempo (spesso si passa da tempi esponenziali a polinomiali) a scapito di un maggiore consumo di memoria.

2. Weighted Interval Scheduling

2.0 Introduzione teorica

Il problema dello scheduling di attività riguarda la gestione di un insieme di attività caratterizzate da un *tempo di inizio*, un *tempo di fine* e un *valore*.

L'obiettivo è quello di determinare un insieme di attività, mutuamente compatibili, il cui valore è massimo. Un insieme di attività si dice mutualmente compatibile nel caso in cui, per ogni attività di tale insieme, nessun'altra attività dell'insieme si sovrappone a questa. Noi lavoreremo con attività ordinate per tempo di fine crescente.

Definiamo inoltre $p(i)$: dato $i \in \{1, \dots, n\}$, **$p(i) = \max\{j \mid j < i \text{ AND } j \text{ è compatibile con } i\}$**

2.1 Pseudocodice

```
M[0] = 0
for i = 1 to n
    M[i] = max{M[p(i)] + vi, M[i-1]}
return M
```


3. Knapsack 0/1 vs Knapsack frazionario.

Differenze concettuali e nell'approccio tramite programmazione

3.0 Definizione del problema Knapsack

Ci sono n oggetti ai quali sono associati un determinato peso ed un determinato valore. Ho uno zaino di capacità in peso C . L'obiettivo è prendere k oggetti tali per cui il valore sia massimo, senza sfondare lo zaino.

3.1 Knapsack 0/1 [programmazione dinamica]

```
for i = 0 to n
    M[i, 0] = 0
for c = 1 to C
    M[0, c] = 0
for i = 1 to n
    for c = 1 to C
        if  $w_i > c$ 
            M[i, c] = M[i-1, c]
        else
            M[i, c] = max{M[i-1, c-wi] + vi, M[i-1, c]}
```

3.1.1 Tempo: $\theta(n \cdot C) \rightarrow$ è pseudopolinomiale!

C non è polinomiale nella dimensione dell'input

3.1.2 Sottostruttura ottima (Knapsack 0/1)

Consideriamo il carico più prezioso che pesa al più W .

Se togliamo l'oggetto j da questo carico, la parte restante deve essere il carico più prezioso, il cui peso non supera $W - w_j$, che si può realizzare con gli $n-1$ oggetti originali (escludendo l'oggetto j).

3.2 Knapsack frazionario [algoritmo greedy]

Differenza con knapsack 0/1: in knapsack 0/1 o prendo un oggetto, o non lo prendo. In knapsack frazionario, invece, posso decidere di prendere solo una frazione di un oggetto.

3.2.1 Tempo: $O(n \cdot \log(n))$, ovvero il tempo necessario per ordinare gli oggetti in base al rapporto valore/peso.

3.2.2 Sottostruttura ottima (Knapsack frazionario)

Se togliamo dal carico ottimo un peso w di *una parte dell'oggetto* j , il restante carico deve essere il carico più prezioso, il cui peso non supera $W - w$, che si può realizzare con gli $n-1$ oggetti originali più $w_j - w$ dell'oggetto j .

4. LCS

4.0 Definizione del problema

Date due sequenze X e Y, trovare la sottosequenza comune più lunga ad esse.

4.1 Variabili necessarie/struttura dati: matrice $M[n, m]$, dove n e m sono le lunghezze delle due stringhe.

$M[i, j]$ = lunghezza della più lunga sottosequenza comune ristretta ai primi i caratteri della stringa X e ai primi j caratteri della stringa Y.

4.2 Equazioni di ricorrenza

- **Caso base:**

- Se $i = 0$ v $j = 0$: $LCS(x_i, y_j) = 0$

- **Passo ricorsivo:**

- Se $x_i = y_j$:
- Se $x_i \neq y_j$:

$$\boxed{LCS(x_{i-1}, y_{j-1}) + 1, \max\{LCS(x_i, y_{j-1}), LCS(x_{i-1}, y_j)\}}$$

espresso ricorsivamente.

nell'algoritmo
equivale a

4.3 Pseudocodice del problema LCS

LCS(X, Y)

$M[n, m]$

for $i = 0$ to n

$M[i, 0] = 0$

for $j = 1$ to m

$M[0, j] = 0$

for $i = 0$ to n

for $j = 1$ to m

if $X[i] == Y[j]$

$M[i, j] = M[i - 1, j - 1] + 1$

else

// MAX in forma estesa

$M[i, j] = M[i - 1, j]$

if $M[i, j - 1] > M[i, j]$

$M[i, j] = M[i, j - 1]$

res = $M[n, m]$

return res

$X = \{a, b, c, d\}$

$Y = \{a, b, c, f\}$

2 SOTTOPROBLEMI

sportarsi nella
TABELLA

$\{a, b, c\}$

$\{a, b, c, c, f\}$

MAX

2 $\begin{cases} \{a, b, c, d\} \\ \{a, b, c, e\} \end{cases}$

4.3 Siano $X = \langle x_1, \dots, x_m \rangle$ e $Y = \langle y_1, \dots, y_n \rangle$ due sequenze e sia $Z = \langle z_1, \dots, z_k \rangle$ una LCS di X e Y.

Scrivere la proprietà di sottostruttura ottima di Z.

1. Se $x_m = y_n$, allora $z_k = x_m = y_n$ e Z_{k-1} è una LCS di X_{m-1} e Y_{n-1}
2. Se $x_m \neq y_n$, allora $z_k \neq x_m$ implica che Z è una LCS di X_{m-1} e Y
3. Se $x_m \neq y_n$, allora $z_k \neq y_n$ implica che Z è una LCS di X e Y_{n-1}

mostra passaggi
Inversi.

5. LGCS

5.0 Differenza LCS-LGCS

La differenza principale riguarda la formulazione del problema.

Sostanzialmente, nel caso LCS, abbiamo tutta l'informazione necessaria,

Ci interessa semplicemente che i caratteri che consideriamo siano uguali, quindi, nel caso questo sia verificato, andiamo ad accodare il carattere alla LCS che stiamo formando, indipendentemente da ciò che avevamo prima.

Con il vincolo aggiuntivo del "crescente" siamo obbligati a sapere cosa c'è alla fine delle soluzioni dei sottoproblemi di taglia minore, per poter capire se è possibile accodare o no un certo elemento comune.

Di conseguenza la formalizzazione del problema richiede di avere un'informazione aggiuntiva, e siamo quindi costretti ad introdurre un *problema ausiliario*.

5.1 Soluzione del problema originale rispetto alle soluzioni di tutti i sottoproblemi del problema ausiliario

$$\max_{1 \leq i \leq m \wedge 1 \leq j \leq n} \{c_{ij}\}$$

5.2 Pseudocodice

```
tmp = 0
for i = 1 to m
  for j = 1 to n
    if  $x_i \neq y_j$ 
       $c_{ij} = 0$ 
    else
      max = 0
      for s = 1 to i - 1
        for t = 1 to j - 1
          if  $x_s \leq x_i$  AND  $c_{st} \geq \text{max}$ 
            max =  $c_{st}$ 
       $c_{ij} = 1 + \text{max}$ 
    if  $c_{ij} > \text{tmp}$ 
      tmp =  $c_{ij}$ 
return tmp
```

tmp contiene la lunghezza di una più lunga sottosequenza crescente di X e Y!

5.3 Tempo: $\theta(n^2 m^2)$

6. Partition

6.0 Definizione del problema

Sono dati n oggetti di valore $v_1, v_2, \dots, v_n$

È possibile dividere questi oggetti in modo equo?

- Se la somma di tutti i numeri è dispari → **no**
- Se la somma è pari ed è uguale a **m**, se possibile devo riuscire a trovare un sottoinsieme di numeri tale che la somma dei loro pesi sia **m/2**

6.1 Spazio occupato

$\theta(n \cdot \log(m)) \rightarrow$ pseudopolinomiale, non è polinomiale nella dimensione dell'input.

Non è vero che posso risolvere il problema in tempo accettabile.

Se $n = 5$ ma uno dei numeri vale 2^{20} , ho una matrice con 2^{19} colonne

È un problema NP-completo: non posso risolverlo in tempo polinomiale [a meno che non si dimostri $P=NP$].

6.2 Algoritmo

```
boolean Partition(V, m)
    if m MOD 2  $\neq$  0:
        return false
    k = m/2
    M[1...n+1, 1...k]
    for j = 1 to k
        M[1, j] = F
    for i = 1 to n
        for j = 1 to k
            if V[i] = j
                M[i, j] = T
    for i = 1 to n
        for j = 1 to k
            if V[i] > j
                M[i, j] = M[i - 1, j]
            else
                M[i, j] = M[i - 1, j] OR M[i - 1, j - V[i]]

    return M[n+1, m/2]
```

6.3 Tempo

$O(n \cdot m/2)$ pseudopolinomiale

7. Subset Sum

7.0 Definizione del problema

Ho un insieme di oggetti, ognuno dei quali possiede un dato valore v_i .

Dato un valore k devo definire, se esiste, l'insieme di elementi la cui somma dei valori sia k .

Questo problema è molto simile al già visto Partition, ma qui il k viene fornito a priori.

Se so risolvere Subset Sum, so risolvere partition con $k=m/2$

$$T(\text{Partition}) = T(\text{Subset Sum}) + \theta(n)$$

7.1 Pseudocodice

```
for i = 0 to n
    M[i, 0] = T
for j = 1 to k
    M[0, j] = F
for i = 0 to n
    for j = 0 to k
        if j == v(i)
            M[i, j] = T
for i = 0 to n
    for j = 0 to k
        if j-v(i) > 0
            M[i, j] = M[i - 1, j] OR M[i - 1, j - v(i)]
        else
            M[i, j] = M[i - 1, j]
```

8. Schema generale algoritmo greedy e tempo di calcolo

Un algoritmo Greedy prende un certo input X e svolge i seguenti passi:

- 1. Definizione dell' insieme delle soluzioni S**
 - a. Spesso $S = \emptyset$, in altri casi possiamo mettere un elemento
 - i. per esempio, il primo nodo da cui partire in un grafo (*sorgente*)
- 2. Scelta e calcolo del parametro di selezione**
- 3. Ordinamento degli elementi in base al parametro di selezione**
- 4. Test iterativo: l'elemento x appartiene o no all'insieme delle soluzioni?**
 - a. In generale l'algoritmo "passa" tutti gli elementi e li aggiunge man mano alla soluzione.

Il tempo impiegato da questa tipologia di algoritmo è pari a $O(n \cdot \log(n))$, sostanzialmente pari al tempo per ordinare gli elementi.

Nel caso in cui non li ordinassi, il tempo sarebbe $O(n^2)$ nel caso pessimo.

Affinché io possa applicare un algoritmo greedy, il problema deve essere ricorsivo!

9. Cos'è un sistema di indipendenza? Cos'è un matroide?

Cosa afferma il teorema di Rado?

9.1 Sistema di indipendenza

Data la coppia $\langle E, F \rangle$ dove E è un insieme finito e F è una famiglia di sottoinsiemi di E , definiamo tale coppia *sistema di indipendenza* se vale la seguente proprietà:

$$\forall A \in F \text{ se } B \subseteq A \Rightarrow B \in F$$

Ovvero: dato un insieme di sottoinsiemi di E chiamato F e preso un qualsiasi elemento A (che è un insieme) da F , in F abbiamo anche tutti i sottoinsiemi di A .

9.2 Matroide

Un sistema di indipendenza è detto matroide se

$$\forall A, B \in F \text{ e } |A| = |B| + 1 \rightarrow \exists a \in A \setminus B \text{ tale che } B \cup \{a\} \in F$$

N.B: NON VUOL DIRE CHE $A = B + \text{UN ELEMENTO}$!

Vuol dire solo che A ha un elemento in più di B !

Data una coppia $\langle E, F \rangle$, questa è un matroide se e solo se valgono **1** e **2**.

Concettualmente:

1. ci dà le soluzioni ammissibili
2. ci dà la soluzione ottima

Anche se A e B non c'entrano niente fra loro, trovo un elemento che aggiunto ad B lo fa "crescere", e ciò che ottengo è ancora un elemento di F .

Al di là di come io scelga A e B , non pregiudico le scelte successive.

È esattamente ciò che voglio dal **greedy**.

9.3 Teorema di Rado

Dato un sistema di indipendenza $\langle E, F \rangle$, le seguenti proposizioni sono equivalenti:

- per ogni funzione peso $w : E \rightarrow \mathbb{R}^+$, l'algoritmo greedy associato fornisce una soluzione ottima
- $\langle E, F \rangle$ è un matroide

N.B: non tutte le funzioni peso sono associabili ad un algoritmo greedy

Prima di proseguire: un riassunto sui grafi

DAG: grafo diretto (orientato) senza cicli, formato da coppie ordinate di punti. Il numero di archi uscenti/entranti in un nodo è detto grado di ingresso/uscita.

Grafo connesso: per ogni coppia di vertici $(u, v) \in V$, esiste un cammino che collega u a v .
Fortemente connesso se il grafo è diretto.

Cammino: sequenza di archi orientati per raggiungere un nodo a partire da un altro nodo. La lunghezza è pari al numero di nodi, meno uno.

Semicammino: cammino che non tiene conto della direzione degli archi.

Distanza: Dato un grafo $G = (V, E)$, per ogni $(u, v) \in V$, definiamo distanza tra i due nodi la lunghezza del cammino (semicammino) che li congiunge (∞ se non esiste)

Ciclo (semiciclo): cammino (semicammino) dove il nodo di partenza è lo stesso di arrivo.

Grafo semplice: grafo non orientato, che non comprende cappi e archi multipli

Grafo completo: è un grafo semplice nel quale ogni vertice è collegato a tutti i vertici rimanenti.

Albero: un grafo non orientato G , tale che:

- G è connesso e aciclico oppure
- G è connesso e $|E| = |V| - 1$

Albero binario: albero nel quale ogni nodo ha $[0, 1, 2]$ figli.

Albero binario senza radice: grafo non orientato, connesso, aciclico, nel quale ogni nodo interno ha al massimo 3 vicini

Foresta: grafo non orientato aciclico.

Una foresta risulta costituita da una unione disgiunta di alberi; questi alberi costituiscono le sue componenti connesse massimali.

Foresta di n alberi: grafo non orientato aciclico che ha n componenti connesse

Grafo completo:

- **Non orientato:** completo sse $|E| = |V| * (|V| - 1) / 2$
- **Orientato:** completo sse $|E| = |V|^2$

Componente (fortemente) connessa: una componente (fortemente) connessa di un grafo (orientato) $G = (V, E)$ è un insieme massimale di vertici $C \subseteq V$ tale che per ogni coppia (u, v) in C esiste un cammino da u a v (ed un cammino da v a u)

10. Rappresentazione in memoria di un grafo

Vi sono due modi per rappresentare un grafo $G = (V, E)$ in memoria:

- Lista di adiacenza
- Matrice di adiacenza

Entrambi i metodi possono essere applicati sia ai grafi orientati che ai grafi non orientati. Solitamente viene preferita la **lista di adiacenza**, perché **permette di rappresentare in modo compatto i grafi sparsi**, ovvero quei grafi dove $|E| \ll |V|^2$.

Usiamo invece la **matrice di adiacenza** nei **grafi densi**, ovvero quei grafi dove $|E|$ è prossimo a $|V|^2$, o quando **dobbiamo dire velocemente se esiste un arco che collega due vertici**.

La **rappresentazione con liste di adiacenza (Adj)** consiste in un array di $|V|$ liste, una per ogni vertice. Per ogni $u \in V$, la lista di adiacenza contiene tutti i vertici v tali che $(u, v) \in E$

La **rappresentazione con matrici di adiacenza**, supponendo che i vertici siano numerati, consiste in una matrice $A = (a_{ij})$ di dimensione $|V| \times |V|$ tale che:

$$a_{i,j} = \begin{cases} 1 & \text{se } (i,j) \in E \\ 0 & \text{altrimenti} \end{cases}$$

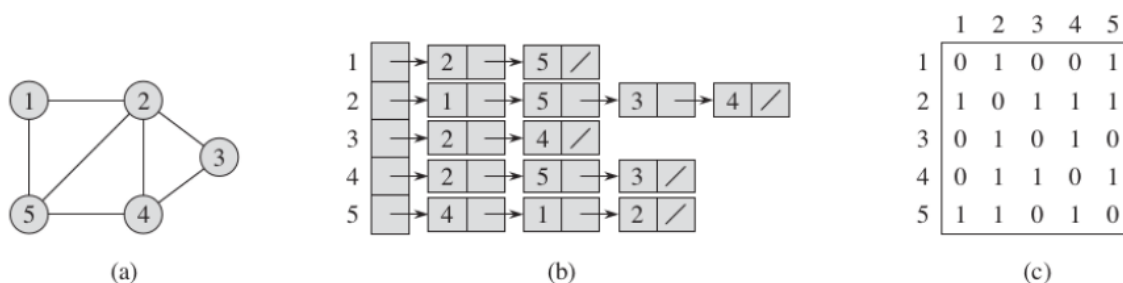


Figure 22.1 Two representations of an undirected graph. (a) An undirected graph G with 5 vertices and 7 edges. (b) An adjacency-list representation of G . (c) The adjacency-matrix representation of G .

11. Algoritmo BFS

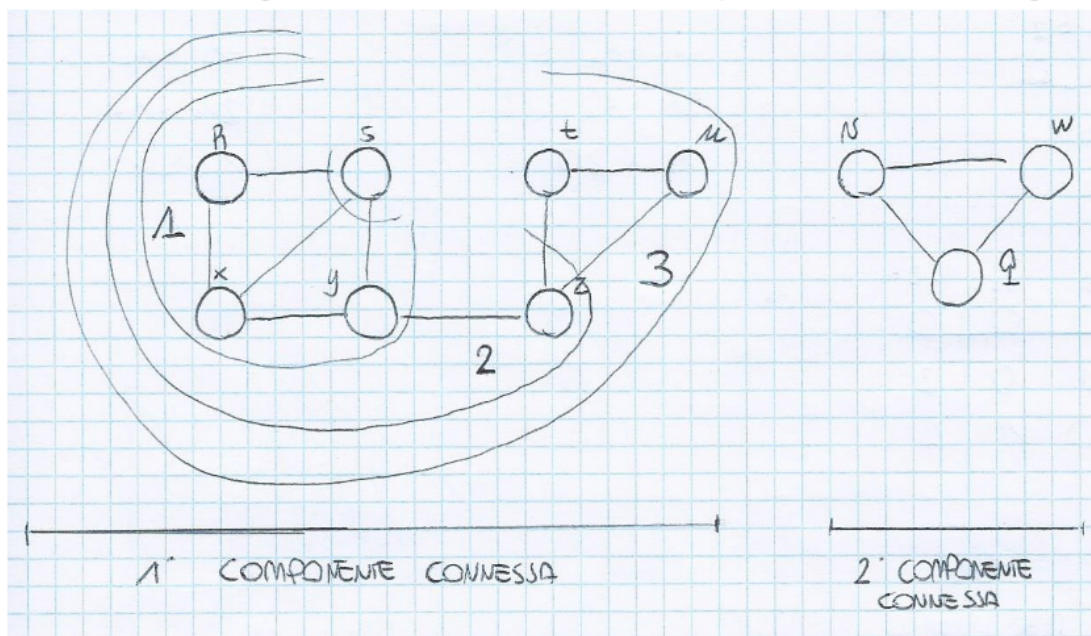
11.1 Qual è l'istanza di BFS?

Un grafo ed un vertice del grafo, definito vertice *sorgente*

11.2 Cosa fa BFS?

BFS (**Breadth-First Search**) è un algoritmo di visita di un grafo *in ampiezza*.

Lo scopo di BFS è quello di **scoprire tutti i vertici raggiungibili, partendo dal vertice sorgente**. La scoperta avviene in ampiezza, nel senso che inizialmente vengono scoperti tutti i vertici a distanza di 1, poi a 2 e così via. Vengono scoperti sempre i **cammini di lunghezza minima**. L'algoritmo visita solo una delle componenti connesse del grafo!



11.3 Cosa sono i colori associati ai vertici?

Usiamo i colori per capire se un vertice è già stato scoperto o no:

- **White** → Vertice non ancora scoperto
- **Gray** → Vertice scoperto ma la lista di adiacenza non è stata scandita del tutto
 - non ho terminato di visitare i vertici adiacenti
 - nella coda tengo i vertici grigi. Mi serve a capire i vertici appena scoperti, da lì andrò "avanti di un livello"
- **Black** → Vertice scoperto ed è stata terminata la scansione della lista di adiacenza

Alla fine dell'esecuzione di BFS, tutti i vertici della componente connessa cui appartiene s saranno colorati di nero.

11.4 Tempo di calcolo di BFS?

$O(|V| + |E|)$

11.4.1 Come mai non è $\theta(|V| + |E|)$?

Perché la parte di inizializzazione viene eseguita su tutti i vertici del grafo ed è quindi $\theta(|V|)$, mentre la parte successiva di visita vera e propria (per intendersi, quella nel ciclo while) esegue *al più* (e non sicuramente!) un numero di operazioni lineare nel numero degli archi di tutto il grafo. Quindi è $O(|E|)$ e $\Omega(1)$.

In conclusione abbiamo $\theta(|V|) + O(|E|) = O(|V| + |E|)$.

11.5 Pseudocodice

BFS(G, s)

```
for ogni u ∈ V \ {s}
    col[u] = WHITE
    d[u]    = ∞
    Π[u]   = NIL      // equivalente a u.π usando un campo di un
                      // oggetto

col[s] = GRAY
d[s] = 0           // la distanza fra la sorgente e se stessa è 0
Π[s] = NIL

Q = ∅              // inizializzo una coda vuota
enqueue(Q, s)

while Q ≠ ∅
    u = dequeue(Q)
    for ogni v ∈ ADJ(u)
        if col[v] = WHITE
            col[v] = GRAY
            d[v] = d[u] + 1
            Π[v] = u
            enqueue(Q, v)
    col[u] = black
```

11.6 Quando estraggo un nodo dalla coda, che succede agli altri?

Come fa l'algoritmo a sapere che non ci sono nodi in sospeso?

L'algoritmo BFS, una volta prelevato un nodo x dalla coda, inizia a scandire la sua lista di adiacenza; una volta che ha terminato tale operazione sul nodo x -eventualmente inserendo nuovi nodi nella coda- controllerà lo stato della coda: se essa risulta vuota, la visita della componente connessa è terminata; altrimenti, preleverà il nodo successivo e ripeterà l'operazione sopra descritta.

Sostanzialmente i nodi nella coda sono "in attesa" della scansione della loro lista di adiacenza.

In generale, per essere sicuri di avere scandito tutti i nodi della componente connessa, basta controllare che, data la componente connessa, non vi siano nodi di colore bianco. Per essere certi di avere visitato tutte le componenti connesse, dovremmo scandire tutti i nodi del grafo e chiamare BFS ogni volta che ne troviamo uno bianco [logica analoga alla chiamata della DFS-VISIT che vedremo fra poco].

11.7 Sottografo dei predecessori: albero BF

La visita in ampiezza costruisce l'**albero BF** che inizialmente contiene la sua radice, il vertice sorgente s . Quando un vertice v bianco viene scoperto durante l'ispezione della lista di adiacenza di un vertice u già scoperto, il vertice v e l'arco (u, v) vengono aggiunti all'albero. Il vertice u è definito **predecessore** o **padre** di v nell'**albero BF**. Poiché un vertice viene scoperto una sola volta, esso ha al più un padre.

Formalmente, per un grafo $G=(V, E)$ con sorgente s , definiamo il **sottografo dei predecessori** di G come $G_\pi = (V_\pi, E_\pi)$, dove

- $V_\pi = \{v \in V : \pi[v] \neq \text{NIL}\} \cup \{s\}$
- $E_\pi = \{(\pi[v], v) : v \in V_\pi \setminus \{s\}\}$

Il sottografo dei predecessori G_π è un **albero BF** se V_π è formato dai vertici raggiungibili da s e, per ogni $v \in V_\pi$, c'è solo un cammino semplice da s a v in G_π - che è anche un cammino minimo da s a v in G .

Un albero BF è effettivamente un albero, perché è connesso e $|E| = |V| - 1$. Gli archi in E_π sono detti **archi dell'albero**.

12. Algoritmo DFS

12.0 Introduzione teorica

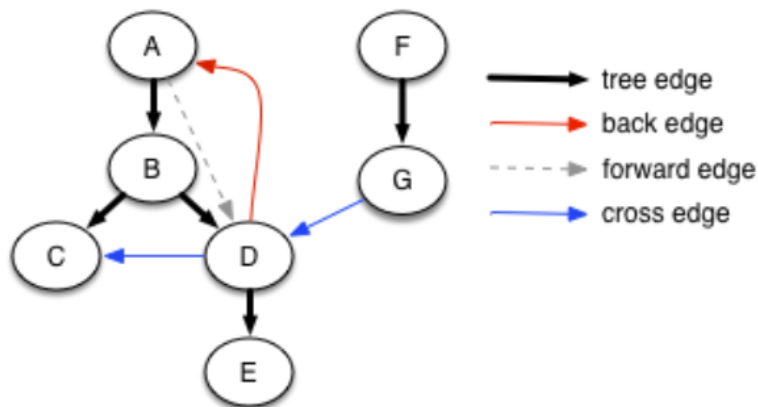
L'algoritmo DFS effettua una visita in profondità di un grafo.

Alla fine dell'esecuzione dell'algoritmo siamo in grado di determinare quante sono le componenti connesse del grafo ed a quale componente connessa appartiene ogni nodo. Quando la componente connessa contenente il nodo sorgente è stata del tutto esplorata, la prima chiamata di DFS-VISIT termina. A questo punto viene chiamata nuovamente sul primo nodo WHITE del grafo, che appartiene ad una componente connessa diversa da quella precedente [quest'ultimo discorso vale solo per i grafi non orientati]. Quindi il numero di chiamate, all'interno di DFS, della procedura DFS-VISIT corrisponde al numero effettivo di componenti connesse.

12.1 Classificazione degli archi in un grafo orientato

Dato un **grafo orientato**, possiamo differenziare gli archi in 4 categorie differenti:

- **Arco dell'albero**
 - Definiamo l'arco (u, v) **arco dell'albero** se v è di colore bianco - quindi se è la prima volta che visitiamo v .
- **Arco all'indietro**
 - Definiamo l'arco (u, v) **arco all'indietro** se v è di colore grigio - quindi v è antenato di u .
- **Arco in avanti**
 - Definiamo l'arco (u, v) **arco in avanti** se v è di colore nero e $d[u] < d[v]$.
N.B.: d = tempo di scoperta (*discovery*), non distanza!
- **Arco trasversale/di attraversamento**
 - Definiamo l'arco (u, v) **arco trasversale/di attraversamento** se v è di colore nero e $d[u] > d[v]$.
N.B.: d = tempo di scoperta (*discovery*), non distanza!



12.2 Classificazione degli archi in un grafo non orientato

Dato un grafo non orientato, possiamo differenziare gli archi in 2 categorie differenti:

- **Arco dell'albero**
 - Definiamo l'arco (u, v) **arco dell'albero** se v è di colore bianco. Quindi se è la prima volta che visitiamo il vertice v
- **Arco all'indietro**
 - Definiamo l'arco (u, v) **arco all'indietro** se v è di colore grigio. Quindi v è antenato di u .
N.B.: $v \neq \Pi[u]$

12.3 Pseudocodice

DFS (G)

```
for ogni  $u \in V$ 
    col[u] = WHITE
     $\Pi[u] = \text{NIL}$  // equivalente a  $u.\pi$ 
time = 0
for ogni  $u \in V$ 
    if col[u] == WHITE
        DFS-VISIT(u)
```

DFS-VISIT(u)

```
col[u] = GRAY
time++
d[u] = time // tempo di scoperta, NON DISTANZA
for ogni  $v \in \text{ADJ}(u)$ 
    if col[v] == WHITE
         $\Pi[v] = u$ 
        DFS-VISIT(v)
time++
f[u] = time // tempo di fine visita
col[u] = BLACK
```

12.4 Che informazioni ricavo da DFS?

Con l'algoritmo DFS posso ricavare informazioni per quanto riguarda le proprietà di un grafo. Per esempio, è facilmente modificabile al fine di sapere:

- il numero di componenti connesse di un grafo
- eventuali proprietà verificabili per ogni componente connessa

12.5 La foresta DFS

Il **sottografo dei predecessori**, data una visita in profondità, può essere formato da più alberi, poiché la visita può essere ripetuta da più sorgenti. Lo definiamo quindi in modo leggermente diverso da quello BFS.

Poniamo $G_\pi = (V, E_\pi)$, dove $E_\pi = \{(\pi[v], v) : v \in V \wedge \pi[v] \neq NIL\}$

Il **sottografo dei predecessori** di una visita in profondità forma una **foresta DF**, composta da vari **alberi DF**. Gli archi in E_π sono detti **archi dell'albero**.

La **foresta DF** è quindi, per grafi non orientati, un grafo non orientato e aciclico (i.e., appunto, una foresta).

12.6 Tempo di calcolo

$\theta(|V| + |E|)$

Motivazione:

- l'inizializzazione (primo ciclo for) richiede $\theta(|V|)$
- la scansione dei vertici in cerca di uno bianco (secondo ciclo for) richiede $\theta(|V|)$
- in DFS-VISIT il ciclo for viene eseguito, per ogni vertice v , $|Adj[v]|$ volte.

$$\sum_{v \in V} |Adj[v]| = \theta(|E|)$$

In totale, abbiamo $\theta(|V| + |E|)$

12.7 Ordinamento topologico con DFS

A livello generale un ordinamento topologico di un **DAG** è un *ordinamento lineare* di tutti i suoi vertici tale che **se G ha un arco (u, v) , allora u appare prima di v nell'ordinamento**.

È bene notare che se il grafo non è aciclico, allora non è possibile ottenere un ordinamento lineare. Un ordinamento topologico di un grafo può essere visto come un ordinamento dei suoi vertici lungo una linea orizzontale in modo che tutti gli archi orientati siano diretti da sinistra a destra.

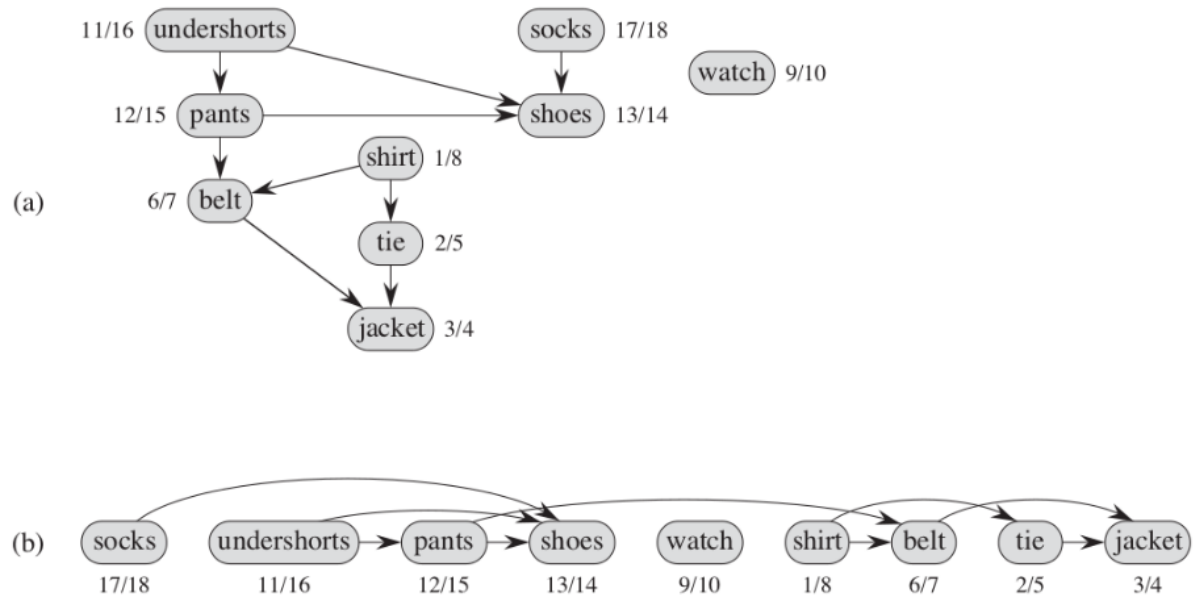
Per quanto riguarda un possibile **ordinamento topologico con DFS** esso è ottenibile guardando, per ogni vertice $v \in V$, il tempo $f[v]$ e ordinando i vertici secondo tale misura in senso decrescente.

TOPOLOGICAL-SORT(G)

1 Chiama DFS(G) per calcolare i tempi di completamento $v.f$ per ciascun vertice

2 Una volta completata l'ispezione di un vertice, inserisce il vertice in testa alla lista concatenata

3 Restituisce la lista concatenata dei vertici



13. Algoritmo di Dijkstra

13.1 Introduzione

L'algoritmo di Dijkstra risolve il problema dei **cammini minimi da sorgente unica**.

Sostanzialmente vogliamo trovare un cammino minimo che va da un dato vertice sorgente $s \in V$ a ciascun vertice $v \in V$ in un grafo orientato pesato $G = (V, E)$ nel caso in cui i **pesi degli archi non** siano **negativi**.

13.2 Proprietà: sottostruttura ottima di un cammino minimo

Come tutti gli algoritmi per cammini minimi, Dijkstra si basa sulla sottostruttura ottima di un cammino minimo, ovvero:

Dato un grafo orientato pesato $G = (V, E)$ con la funzione peso $w : E \rightarrow R$, sia $p = [v_1, v_2, \dots, v_k]$ un cammino minimo dal vertice v_1 al vertice v_k e, per qualsiasi i e j tali che $1 \leq i \leq j \leq k$, sia $p_{ij} = [v_i, v_{i+1}, \dots, v_j]$ il sottocammino di p dal vertice v_i al vertice v_j . Allora p_{ij} è un cammino minimo da v_i a v_j .

13.3 Stima del cammino minimo, predecessore, procedura di rilassamento

Un altro concetto fondamentale è l'istituzione di due attributi per ogni $v \in V$.

- **v.d = stima del cammino minimo**
Inizialmente posto a ∞ , questo attributo serve a definire un **limite superiore** per il peso del cammino minimo dal nodo sorgente **s** al nodo **v**.
Ovviamente $s.d = 0$
- **v. π** = utilizzato per indicare il **predecessore di v**

Questa definizione serve per introdurre il concetto di **rilassamento** di un arco (u, v) che consiste nel *verificare se, passando per u, è possibile migliorare il cammino minimo per v precedentemente trovato e, in caso affermativo, modificare gli attributi sopra con i nuovi valori*.

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2    v.d = u.d + w(u, v)
3    v. $\pi$  = u
```

Spesso, viene utilizzata INITIALIZE-SINGLE-SOURCE(G, v). È una funzione utile per fare in modo che l'algoritmo abbia come "sorgente" il vertice v .

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for ogni v  $\in$  V
2    v.d =  $\infty$ 
3    v. $\pi$  = NIL
4  s.d = 0
```

13.3.1 Proprietà dei cammini minimi e del rilassamento

- **Disuguaglianza triangolare**
 - per qualsiasi arco $(u, v) \in E$, si ha $\delta(s, v) \leq \delta(s, u) + w(u, v)$
- **Proprietà del limite superiore**
 - per tutti i vertici $v \in V$, si ha sempre $v.d \geq \delta(s, v)$ e, una volta che il limite superiore $v.d$ assume il valore $\delta(s, v)$, esso non cambia più
- **Proprietà dell'assenza di un cammino**
 - Se non c'è un cammino da s a v , allora si ha sempre $v.d = \delta(s, v) = \infty$
- **Proprietà della convergenza**
 - se $s \Rightarrow u \rightarrow v$ è un cammino minimo in G per qualche $u, v \in V$ e se $u.d = \delta(s, u)$ in un istante qualsiasi prima del rilassamento dell'arco (u, v) , allora $v.d = \delta(s, v)$ in tutti gli istanti successivi
- **Proprietà del rilassamento del cammino**
 - Se $p = \langle v_0, v_1, \dots, v_k \rangle$ è un cammino minimo da $s = v_0$ a v_k e gli archi di p vengono rilassati nell'ordine $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, allora $v_k.d = \delta(s, v_k)$. Questa proprietà è soddisfatta indipendentemente da altri passi di rilassamento che vengono effettuati, anche se sono interposti fra i rilassamenti degli archi di p
- **Proprietà del sottografo dei predecessori**
 - Una volta che $v.d = \delta(s, v)$ per ogni $v \in V$, il sottografo dei predecessori è un albero di cammini minimi radicato in s

13.4 Pseudocodice

```
DIJKSTRA( $G, w, s$ ) // Grafo, funzione peso, vertice sorgente
1 INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  $S = \emptyset$  // insieme vuoto
3  $Q = G.V$  // coda
4 while  $Q \neq \emptyset$ 
5      $u = \text{EXTRACT-MIN}(Q)$ 
6      $S = S \cup \{u\}$ 
7     for ogni vertice  $v \in \text{Adj}[u]$ 
8         RELAX( $u, v, w$ )
```

13.5 Spiegazione algoritmo

- 1: inizializza i valori d e π per ogni vertice, per s ho $s.d = 0$
- 2: inizializza l'insieme S come insieme vuoto
- 3: inizializza la coda in modo che questa contenga tutti i vertici V
- 5: in questa fase viene estratto il vertice con $v.d$ minore, al primo giro viene estratto s in quanto $s.d = 0$, tutti gli altri hanno $v.d = \infty$.
- 6: u viene aggiunto all'insieme delle soluzioni
- 7-8: viene effettuato il *rilassamento per ogni arco (u, v) che esce da u* e, se il cammino minimo che arriva a v può essere migliorato passando per u , vengono aggiornati la stima $v.d$ e il predecessore $v.\pi$

13.6 Perché segue un approccio greedy?

Perché l'algoritmo di Dijkstra sceglie sempre il vertice "più leggero" o "più vicino" in $V \setminus S$ da aggiungere all'insieme S . Vi è un'analogia con il criterio per l'ordinamento negli algoritmi greedy.

13.7 Teorema sulla correttezza dell'algoritmo di Dijkstra

L'algoritmo di Dijkstra, eseguito su un grafo orientato pesato $G = (V, E)$ con funzione peso non negativa w e sorgente s , termina con $u.d = \delta(s, u)$ per tutti i vertici $u \in V$.

NB: Con $\delta(s, u)$ indichiamo il cammino minimo $s \rightarrow u$

13.8 Tempo d'esecuzione

- $O(E + V \cdot \log V)$ utilizzando heap di Fibonacci per estrarre velocemente il nodo a distanza minore nella coda
- $O(V^2)$ altrimenti (implementazione naïve)

14. Bellman-Ford

14.0 Introduzione

L'algoritmo di Bellman-Ford risolve il problema dei **cammini minimi da sorgente unica** nel caso generale in cui i **pesi degli archi possono essere negativi**. Svolge quindi lo stesso lavoro di Dijkstra, ma senza il vincolo che i pesi siano tutti non negativi.

14.1 Input

Un grafo orientato $G=(V, E)$, un vertice sorgente s e una funzione peso $w: E \rightarrow \mathbb{R}^*$

14.2 Output

Valore booleano che indica se esiste oppure no un ciclo di peso negativo che è raggiungibile dalla sorgente. Se tale ciclo non esiste, l'algoritmo fornisce i cammini minimi e i loro pesi. L'algoritmo restituisce *true* *se e solo se* il grafo non contiene cicli di peso negativo che sono raggiungibili dalla sorgente.

14.3 Analisi dell'algoritmo

Così, come Dijkstra, anche Bellman-Ford utilizza il rilassamento, riducendo progressivamente il valore stimato $v.d$ per il peso di un cammino minimo dalla sorgente s ad ogni nodo v del grafo.

In Bellman-Ford il rilassamento viene eseguito $|V|-1$ volte per arco indipendentemente dalla morfologia del grafo, mentre in Dijkstra il rilassamento viene eseguito esattamente una volta per arco. Inoltre, Dijkstra segue un approccio *greedy*, Bellman Ford no!

14.4 Tempo

$O(|V| \cdot |E|)$: ciascun arco viene rilassato $|V|$ volte [più un $O(|E|)$ finale per il controllo sull'assenza di cicli negativi].

14.5 Pseudocodice

```
Bellman-Ford( $G, s$ )
1 INITIALIZE-SINGLE-SOURCE( $G, s$ )
2 for  $i = 1$  to  $|V|-1$ 
3     for ogni arco  $(u, v) \in E$ 
4         RELAX( $u, v, w$ )
5 for ogni arco  $(u, v) \in E$ 
6     if  $v.d > u.d + w(u, v)$ 
7         return FALSE
8 return TRUE
```

15. Algoritmi per la costruzione di alberi di copertura minimi: Kruskal e Prim

15.0 Introduzione del problema

Il problema della costruzione di alberi minimi **riguarda solo i grafi** $G = (V, E)$ **non orientati**, dotati di una funzione peso $w: E \rightarrow \mathbb{R}$.

L'obiettivo è trovare un albero di connessione minimo per il grafo, ovvero, dato un grafo con un determinato numero di archi, determinare un sottoinsieme di archi tale per cui i cammini tra i vari vertici sono minimi.

I due algoritmi che analizzeremo, Kruskal e Prim, sono entrambi identificabili come *greedy*.

15.0.1 Definizioni di taglio, arco sicuro e arco leggero

Taglio

Un taglio $(S, V \setminus S)$ di un grafo non orientato $G = (V, E)$ è una partizione di V . In altre parole, dato un insieme S dividiamo i vertici V in due insiemi: appartenenti a S e non appartenenti a S .

Arco sicuro, arco leggero

Sia $G = (V, E)$ un grafo connesso, non orientato e pesato. Siano inoltre:

- $A \subseteq E$ un sottoinsieme di archi di un MST (albero di copertura minimo, *minimum spanning tree*)
- $(S, V \setminus S)$ un taglio definito su G che rispetta A , ovvero non ci sono archi in A che attraversano il taglio (attraversare significa che un arco unisce due vertici: uno in S e l'altro in $V \setminus S$).
- $(u, v) \in E$ un arco di peso minimo che attraversa il taglio definito su G (un *arco leggero*)

allora (u, v) è un arco sicuro per A , ovvero: se aggiungiamo l'arco (u, v) ad A , quest'ultimo continua a rappresentare un sottoinsieme di archi di un MST.

Note:

- in generale, **un arco è leggero rispetto ad una determinata proprietà se è quello di peso minimo fra tutti gli archi che rispettano quella proprietà**
- in caso di parità di peso, abbiamo più archi leggeri

15.0.2 Generic MST

GENERIC-MST (G, w) //grafo, funzione peso

```
1  $A = \emptyset$ 
2 while  $A$  non forma albero di connessione
3     trova un arco  $(u, v)$  sicuro per  $A$ 
4      $A = A \cup \{(u, v)\}$ 
5 return  $A$ 
```

15.0.3 Analisi sull'invariante di ciclo:

Inizializzazione: Dopo la riga 1, l'insieme A soddisfa banalmente l'invariante di ciclo

Conservazione: Il ciclo nelle righe 2-4 conserva l'invariante perché aggiunge solo archi sicuri

Conclusione: Tutti gli archi aggiunti ad A si trovano in un albero di connessione minimo, quindi l'insieme A restituito deve essere un albero di connessione minimo.

15.1 Algoritmo di Kruskal

L'algoritmo di Kruskal trova un **arco sicuro da aggiungere alla foresta in costruzione** scegliendo, fra tutti gli archi che collegano due alberi qualsiasi nella foresta, un arco (u, v) di **peso minimo**. L'algoritmo di Kruskal è un algoritmo goloso in quanto, a ogni passo, aggiunge alla foresta un arco con il minor peso possibile. La struttura dati utilizzata è un *insieme disgiunto di elementi*.

La differenza sostanziale con Prim è che, mentre Prim -in fase d'esecuzione- costruisce un albero, Kruskal lavora costruendo una foresta, e solo alla fine dell'esecuzione siamo certi di avere un MST.

15.1.1 Pseudocodice

```
MST-KRUSKAL(G, w) // Grafo, funzione peso
1 A =  $\emptyset$ 
2 for ogni  $v \in G.V$ 
3     MAKE-SET(v)
4 ordina gli archi in senso non decrescente rispetto a w
5 for ogni  $(u, v) \in G.E$  // Parto dall'arco più "economico"
6     if FIND-SET(u)  $\neq$  FIND-SET(v)
7         A = A  $\cup$  { (u, v) }
8         UNION(u, v)
9 return A
```

15.1.2 Spiegazione pseudocodice

- 1: Viene inizializzato l'insieme delle soluzioni come insieme vuoto
- 2-3: Vengono creati tanti alberi [i.e. insiemi di un solo elemento] quanti sono i vertici del grafo
- 4: Gli archi vengono ordinati in senso non decrescente
- 5: Si cicla sugli archi ordinati
- 6-8: Dati i 2 vertici di un arco, si verifica se essi siano già parte dello stesso insieme; se così non è, vengono aggiunti all'insieme delle soluzioni, venendo uniti nello stesso insieme [albero].
- 9: Si ritorna la soluzione

15.1.3 Tempo di esecuzione

Il tempo di esecuzione dell'algoritmo di Kruskal dipende dall'implementazione della struttura dati per gli insiemi disgiunti. In generale il tempo è pari a **$O(E \cdot \log V)$** .

15.2 Algoritmo di Prim

L'algoritmo di Prim opera in maniera molto *simile all'algoritmo di Dijkstra*. L'algoritmo di Prim ha la proprietà che **gli archi dell'insieme delle soluzioni A formano sempre un albero singolo, non una foresta**. La costruzione avviene dato un nodo che potremmo definire "nodo sorgente". Ad ogni passo viene aggiunto all'albero A un arco leggero che collega A con un vertice isolato; ovviamente, questo vertice isolato non deve fare già parte di A. Anche questo algoritmo è di natura *greedy*, in quanto l'albero cresce includendo a ogni passo un arco che contribuisce con la quantità più piccola possibile a formare il peso dell'albero. Prim utilizza una **coda di min-priorità** come struttura dati. Per ogni vertice v , l'attributo $v.key$ è il peso minimo di un qualsiasi arco che connetta v a un vertice dell'albero.

15.2.1 Pseudocodice

```
MST-PRIM( $G, w, r$ ) // Grafo, funzione peso, sorgente
1 for ogni  $u \in G.V$ 
2      $u.key = \infty$ 
3      $u.\pi = NIL$ 
4  $r.key = 0$ 
5  $Q = G.V$ 
6 while  $Q \neq \emptyset$ 
7      $u = \text{EXTRACT-MIN}(Q)$  // qui si aggiunge l'arco all'albero
8     for ogni  $v \in \text{Adj}[u]$ 
9         if  $v \in Q$  and  $w(u, v) < v.key$ 
10              $v.key = w(u, v)$ 
11              $v.\pi = u$ 
```

15.2.2 Spiegazione pseudocodice

1-3: Per ogni vertice del grafo imposto i valori key e π
4: imposto il valore $key = 0$ al nodo "sorgente"
5: Inserisco nella coda tutti i vertici del grafo
6: Eseguo tutte le operazioni delle righe seguenti finché la coda non è vuota
7: Estraggo dalla coda il vertice con key minore; alla prima iterazione sarà il vertice r
8-11: Data la lista di adiacenza del vertice appena estratto, verifico se i vertici da esso raggiungibili fanno parte della coda, ovvero non sono ancora stati inseriti nell'insieme delle soluzioni. Verifico anche che la chiave di questi vertici sia maggiore rispetto al "rilassamento" per u . Se le due condizioni sono verificate, aggiorno i valori key e π di v .

15.2.3 Invarianti e strutture dati

Durante l'esecuzione dell'algoritmo, l'insieme A è mantenuto implicitamente come invariante nella forma:

$$A = \{(v, v.\pi) : v \in V - \{r\} - Q\}$$

Quando l'algoritmo termina, la coda min-priorità Q è vuota; l'albero di connessione minimo A per G è quindi:

$$A = \{(v, v.\pi) : v \in V - \{r\}\}$$

r è escluso in quanto non ha predecessore: r sarà predecessore di altri vertici.

15.2.4 Tempo di esecuzione

Il tempo di esecuzione dipende dall'implementazione della coda, possiamo affermare che il tempo di esecuzione è pari a $O(E + V \log V)$ utilizzando heap di Fibonacci (come Dijkstra).

16. Floyd-Warshall

16.0 A cosa serve l'algoritmo?

L'algoritmo di Floyd-Warshall serve per calcolare il *cammino minimo da i a j* per ogni coppia di vertici (i, j) del grafo su cui viene eseguito.

16.1 Input e output

Input: matrice che contiene il peso dell'arco tra ogni coppia di vertici di un grafo

Output: matrice D dei pesi dei cammini minimi per ogni coppia di vertici (i, j), nella quale $D_{ij} = \delta(i, j)$

16.2 Perché è un algoritmo di programmazione dinamica?

Possiamo definirlo algoritmo di programmazione dinamica in quanto non andremo mai a ricalcolare valori già calcolati, che verranno memorizzati in una matrice. Sfruttiamo una tecnica bottom-up.

16.3 Equazione di ricorrenza

$$d_{i,j}^{(k)} = \begin{cases} W_{ij} & \text{if } k = 0 \\ \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\} & \text{if } k > 0 \end{cases}$$

16.4 Tempo di calcolo: $\theta(n^3)$

16.5 Spazio: $\theta(n^2)$

16.6 Pseudocodice:

Floyd-Warshall(W)

 D = W

 for k = 1 to n

 for i = 1 to n

 for j = 1 to n

 if D[i, j] > D[i, k] + D[k, j]

 D[i, j] = D[i, k] + D[k, j]

return D

16.7 Metodo per costruire i cammini minimi

Ci sono vari metodi per costruire i cammini minimi nell'algoritmo di Floyd-Warshall. Uno consiste nel calcolare la matrice D dei cammini minimi e poi costruire la matrice dei predecessori Π dalla matrice D . Questo metodo può essere implementato con tempo $O(n^3)$.

$\pi_{ij}^{(k)}$ è definito come il predecessore del vertice j in un cammino minimo dal vertice i , i cui vertici intermedi si trovano tutti nell'insieme $\{1, \dots, k\}$.

Caso base: $k=0$

$$\pi_{ij}^{(0)} = \begin{cases} NIL & \text{se } i = j \vee w_{ij} = \infty \\ i & \text{se } i \neq j \vee w_{ij} < \infty \end{cases}$$

Passo ricorsivo: $k>0$

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{se } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \pi_{kj}^{(k-1)} & \text{se } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases}$$

16.8 Chiusura transitiva di un grafo orientato

Dato un grafo orientato $G = (V, E)$ la chiusura transitiva di G è definita come il grafo

$G^* = (V, E^*)$ dove:

$E^* = \{(i, j) \text{ t.c. esiste un cammino dal vertice } i \text{ al vertice } j \text{ in } G\}$

Un metodo per calcolare la chiusura transitiva tramite l'algoritmo Floyd-Warshall è sostituire le operazioni aritmetiche $+$ e $\min$ con $\wedge$ e $\vee$.

Definiamo t_{ij}^k pari a 1 se esiste un cammino nel grafo G dal vertice i al vertice j con vertici intermedi $\in \{1, \dots, k\}$, pari a 0 negli altri casi. Costruiamo la chiusura transitiva $G^* = (V, E^*)$ ponendo l'arco (i, j) in E^* se e solo se t_{ij}^n è uguale a 1.

16.8.1 Caso base

$$t_{i,j}^0 = \begin{cases} 0 & \text{if } i \neq j \wedge (i, j) \notin E \\ 1 & \text{if } i = j \vee (i, j) \in E \end{cases}$$

16.8.2 Caso generico

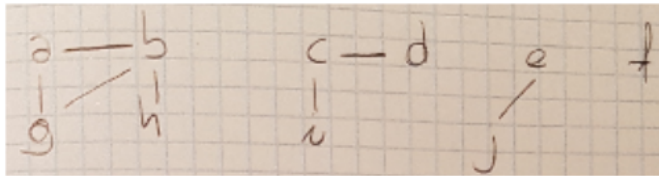
$$t_{i,j}^{(k)} = t_{i,j}^{(k-1)} \vee (t_{i,k}^{(k-1)} \wedge t_{k,j}^{(k-1)})$$

17. Strutture dati per insiemi disgiunti

Funzioni disponibili

- **union(x_i, x_j)**: unisce i due insiemi a cui appartengono x_i e x_j .
Chiama la `find_set` su entrambi e fonde i due insiemi a cui appartengono x_i e x_j .
Posso scegliere io il nuovo rappresentante, l'importante è che alla fine vi sia uno e un solo rappresentante che identifichi l'insieme appena creato.
Molto spesso il nuovo rappresentante è uno fra `find_set( $x_i$ )` e `find_set( $x_j$ )`
- **find_set(x_i)**: un elemento dell'insieme di partenza X che viene eletto come rappresentante dell'insieme.
`find_set( $x_i$ )` prende un elemento $x_i \in X$ e restituisce un elemento $y_i \in X$ che è il rappresentante dell'insieme a cui appartiene x_i
Per rappresentare un insieme uso -appunto- il suo rappresentante!
- **make_set(x_i)**: prende in ingresso $x_i \in X$ e crea insieme di un solo elemento, con x_i rappresentante dell'insieme (che è fatto di un solo elemento).

Trovare il numero di componenti connesse di un grafo $G=(V, E)$ tramite le 3 funzioni definite sopra



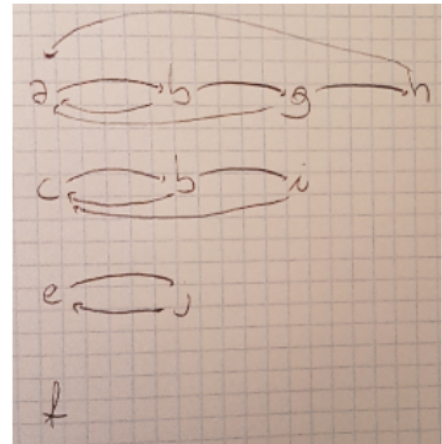
```
for ogni  $v \in V$ 
    make_set( $v$ )
for ogni  $(v_i, v_j) \in E$ 
     $R_1 = \text{find\_set}(v_i)$ 
     $R_2 = \text{find\_set}(v_j)$ 
    if  $R_1 \neq R_2$ 
        union( $R_1, R_2$ )    // assumiamo che il rappresentante del
                           // nuovo insieme sia  $R_1$ 
```

Quanto tempo impiegano `find_set`, `make_set`, `union`?

Usiamo delle liste. In ogni lista metto gli elementi di un insieme. Inoltre faccio puntare ogni elemento all'elemento in testa alla lista (*rappresentante*).

union: ultimo elemento della prima lista punta al primo della seconda lista e aggiorno il rappresentante degli elementi della seconda lista.

make_set richiede tempo **costante**
find_set richiede tempo **costante**
union richiede tempo **ammortizzato $O(n)$**



Come possiamo migliorare `union`?

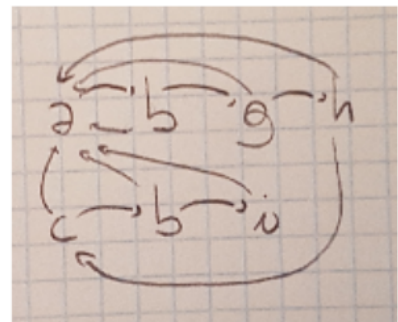
Possiamo fare fino a n union, ed ognuna può richiedere fino a n passi.

Caso pessimo: $\frac{(n-1) \cdot n}{2} \rightarrow O(n^2)$

Lavoro in maniera più intelligente:

di ogni lista tengo non solo il puntatore in testa, ma anche la lunghezza.

Quando faccio la union, faccio **union(più lunga, più corta)**, in modo che sia la più corta ad essere aggiornata sulla più lunga.



Assumiamo di avere 100 elementi e lavoriamo per potenze di 2 (ovvero, eseguiamo sempre `union` su insiemi di egual cardinalità). Arriviamo al limite superiore, 64, in 7 passaggi - arrotondamento per eccesso di $\log_2(100)$

1+1 → 1
2+2 → 2
4+4 → 4
8+8 → 8
16+16 → 16
32+32 → 32
64+64 → 64

Lo spazio occupato rimane lineare, mentre il tempo richiesto si abbassa a $O(n \cdot \log(n))$ nel caso pessimo. L'*euristica dell'unione pesata* mantiene un costo in tempo $\Omega(n)$ per la singola operazione `union`, ma riduce il tempo complessivo richiesto da n union.

In conclusione, in ogni elemento della lista abbiamo salvati:

- valore
- puntatore alla testa (*rappresentante*)
- puntatore alla coda (solo il rappresentante)
- puntatore al *next*

Foreste di insiemi disgiunti

Un'altra implementazione prevede l'utilizzo di alberi al posto delle liste.

Si tiene traccia dell'altezza dell'albero. Il puntatore a radice porta implicitamente con sé l'altezza dell'albero (*rango*).

make_set → Albero rappresentato da un nodo.

Tempo costante

find_set → Il rappresentante è la radice, devo risalire l'albero (puntatore a parent)

Tempo ammortizzato: $O(n)$.

Nello svolgere la *find_set* possiamo applicare la *compressione dei cammini*, facendo diventare figli della radice tutti i nodi, in modo che le operazioni eseguite successivamente avverranno tutte in tempo costante

union → Unisco le radici. **Tempo costante.**

Euristica dell'unione per rango: si fa diventare l'albero di profondità (rango) minore figlio dell'albero di profondità (rango) maggiore.

Se i due ranghi sono uguali bisogna aumentare il rango di tutto l'albero

Foresta → meglio se ho più union che find_set

Liste → meglio se ho più find_set che union

Prestazioni

Sia m il numero totale di operazioni svolte.

In generale, i tempi complessivi sono del tipo $O(m \cdot \log(n))$

Tramite le varie euristiche (compressione del cammino, unione per rango), si ottengono tempi di circa $O(k \cdot n)$, con $k \leq 4$.

18. Reti di flusso e Ford-Fulkerson

18.0 Introduzione teorica

Una **rete di flusso** $G = (V, E)$ è un grafo orientato in cui ogni arco $(u, v) \in E$ ha una **capacità** non negativa $c(u, v) \geq 0$. Si richiede inoltre che se E contiene un arco (u, v) , allora non esiste un arco (v, u) nella direzione opposta. Se $(u, v) \notin E$, allora per convenienza definiamo $c(u, v) = 0$.

Distinguiamo due vertici in una rete di flusso: una **sorgente** s e un **pozzo** t . Per comodità, supponiamo che ogni vertice giaccia in qualche cammino dalla sorgente al pozzo.

18.1 Definizione formale

Sia $G = (V, E)$ una rete di flusso con una funzione capacità c . Sia s la sorgente della rete e sia t il pozzo. Un **flusso** in G è una funzione a valori reali $f : V^2 \rightarrow \mathbb{R}$ che soddisfa la seguenti due proprietà:

- **Vincolo sulla capacità:** per ogni $u, v \in V$, si richiede che $0 \leq f(u, v) \leq c(u, v)$. Questo vincolo dice che il flusso da un vertice all'altro deve essere non negativo e non deve superare la capacità prefissata.
- **Conservazione del flusso:** per ogni $u \in V - \{s, t\}$, si richiede che

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$$

Questa proprietà dice che il flusso totale che entra in un vertice, diverso dalla sorgente o dal pozzo, deve essere uguale al flusso totale che esce da quel vertice. Informalmente "il flusso entrante è uguale al flusso uscente".

Quando $(u, v) \notin E$, non ci può essere flusso da u a v , e $f(u, v) = 0$.

La quantità $f(u, v)$ è detta **flusso** dal vertice u al vertice v . Il **valore** $|f|$ di un flusso è definito come

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$$

Ovvero il flusso totale che esce dalla sorgente meno il flusso totale che entra nella sorgente.

NB: Normalmente una rete di flusso non ha archi entranti nella sorgente, e il flusso che entra nella sorgente, dato da $\sum_{v \in V} f(v, s)$ sarà 0.

18.2 Flusso massimo

Nel problema di **flusso massimo** sono dati una rete di flusso G , una sorgente s e un pozzo t e si vuole trovare un flusso di valore massimo fra tutti i possibili.

18.3 Algoritmo di Ford-Fulkerson

Solitamente viene definito "metodo di **Ford-Fulkerson**" in quanto esso ammette varie implementazioni con tempi di esecuzioni diversi.

Il metodo di **Ford-Fulkerson** dipende da tre importanti idee:

- Reti residue
- Cammini aumentanti
- Tagli

18.3.1 Reti residue

Data una rete di flusso G e un flusso f , la rete residua G_f è una rete composta da archi con capacità che rappresentano come possiamo modificare il flusso negli archi di G . *Un arco della rete di flusso può accettare una quantità di flusso aggiuntivo pari alla capacità dell'arco meno il flusso su tale arco.* Se tale valore è positivo, inseriamo questo arco in G_f con una *capacità residua* pari a:

$$c_f(u, v) = c(u, v) - f(u, v)$$

Gli archi (u, v) il cui flusso è uguale alla loro capacità hanno $c_f(u, v) = 0$ e non si trovano in G_f . Per rappresentare una possibile riduzione di flusso positivo su $f(u, v)$ in un arco di G , poniamo un arco (v, u) in direzione opposta in G_f . Questi archi possono al più annullare il flusso su (u, v) e ci permettono di *ridurre* il flusso nell'arco.

Supponiamo di avere una rete di flusso $G = (V, E)$ con una sorgente s e un pozzo t .

Sia f un flusso in G . Consideriamo una coppia di vertici $(u, v) \in V$.

Definiamo la **capacità residua** $c_f(u, v)$ in questo modo:

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{se } (u, v) \in E \\ f(v, u) & \text{se } (v, u) \in E \\ 0 & \text{negli altri casi} \end{cases}$$

18.3.2 Cammini aumentanti

Data una rete di flusso $G = (V, E)$ con flusso f , un **cammino aumentante** p è un cammino semplice da s a t nella rete residua G_f .

Per la definizione di rete residua, possiamo aumentare il flusso di un dato arco (u, v) in un cammino aumentante fino a $c_f(u, v)$, senza violare il vincolo sulla capacità.

La quantità massima di cui può crescere il flusso in ogni arco di un **cammino aumentante** p è detta **capacità residua** di p ed è espressa da:

$$c_f(p) = \min\{c_f(u, v) : (u, v) \text{ è in } p\}$$

18.3.2.1 Lemma

Sia $G = (V, E)$ una rete di flusso, sia f un flusso in G e sia p un cammino aumentante di G_f . Definiamo una funzione $f_p: V^2 \rightarrow \mathbb{R}$ come:

$$f_p(u, v) = \begin{cases} c_f(p) & \text{se } (u, v) \in p \\ 0 & \text{negli altri casi} \end{cases}$$

Allora f_p è un flusso in G_f con valore $|f_p| = c_f(p) > 0$.

18.3.3 Tagli

Un **taglio** (S, T) della rete di flusso $G = (V, E)$ è una partizione di V in S e $T = V \setminus S$ tale che $s \in S$ e $t \in T$. La definizione è simile a quella presentata per gli alberi di connessione minimi, tuttavia qui vi è la differenza per quanto riguarda il grafo, infatti utilizziamo un grafo orientato.

Se f è un flusso, allora il **flusso netto** $f(S, T)$ attraverso il taglio (S, T) è definito come:

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$$

La **capacità** del taglio (S, T) è:

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$$

Un **taglio minimo** di una rete di è un taglio la cui capacità è minima rispetto a tutti i tagli della rete.

18.4.0 Metodo di base di Ford-Fulkerson

```
Ford-Fulkerson Method(G, s, t) \\Grafo, sorgente, pozzo
1 Inizializza flusso f a 0
2 While esiste un cammino aumentante p nella rete residua Gf
3     Aumenta il flusso f su p
4 return f
```

18.4.1 Un'implementazione di Ford-Fulkerson

In ogni iterazione del metodo di **Ford-Fulkerson** troviamo qualche cammino aumentante p e utilizziamo p per modificare il flusso f .

La seguente implementazione del metodo **calcola il flusso massimo in una rete di flusso** $G = (V, E)$ aggiornando l'attributo del flusso $(u, v).f$ per ogni arco $(u, v) \in E$.

Se $(u, v) \notin E$, assumiamo che $(u, v).f = 0$. Supponiamo anche che le capacità degli archi vengano fornite con la rete di flusso e che $c(u, v) = 0$ se $(u, v) \notin E$.

```
Ford-Fulkerson (G, s, t) //Grafo, sorgente, pozzo
1 for ogni arco (u, v) ∈ G.E
2     (u, v).f = 0
3 while esiste un cammino p da s a t nella rete residua Gf
4     cf(p) = min {cf(u, v) : (u, v) è in p}
5     for ogni arco (u, v) in p
6         if (u, v) ∈ E
7             (u, v).f = (u, v).f + cf(p)
8         else
9             (v, u).f = (v, u).f - cf(p)
```

18.5 Spiegazione algoritmo

1-2: Inizializzazione del flusso f a 0.

3-8: troviamo ripetutamente un cammino aumentante p in G_f e incrementiamo il flusso f lungo p della capacità residua $c_f(p)$.

6-8: aggiorniamo il flusso in ciascun caso, sommando il flusso quando l'arco residuo è un arco originale e sottraendo il flusso nell'altro caso.

18.6 Tempo di esecuzione

Il tempo di esecuzione dipende dal modo in cui viene trovato il cammino aumentante p , possiamo dire che richiede un tempo pari a $O(E \cdot |f^*|)$

Se f^* denota un flusso massimo nella rete trasformata, una semplice implementazione di Ford-Fulkerson esegue il ciclo while al massimo $|f^*|$ volte, dal momento che il valore del flusso aumenta di *almeno* 1 ad ogni iterazione.

19. NP completezza

19.0 Cenni teorici

Lo studio della **NP-completezza** inizia formalizzando il concetto dei problemi risolvibili in tempo polinomiale.

In primo luogo, sebbene sia ragionevole considerare intrattabile un problema che richiede $\theta(n^{100})$, ci sono pochissimi problemi nella pratica che richiedono un tempo polinomiale così alto. Una volta che viene scoperto un algoritmo con un tempo **polinomiale**, spesso seguono altri algoritmi più efficienti.

In secondo luogo, per molti modelli di calcolo, un problema che può essere risolto in tempo polinomiale in un modello può essere risolto sempre in tempo polinomiale in un altro modello.

In terzo luogo, la classe dei problemi risolvibili in tempo polinomiale ha delle interessanti proprietà di chiusura, in quanto i polinomi sono chiusi rispetto all'addizione, alla moltiplicazione e alla composizione.

19.0.1 Problema di decisione

Un problema di decisione è un problema tale per cui la risposta è *si* o *no*.

19.0.2 Classe di complessità

Nella teoria della complessità computazionale, una classe di complessità è un insieme di problemi di una certa complessità.

Un esempio tipico di definizione di classe di complessità ha la forma:

l'insieme di problemi che, se esiste la soluzione, possono essere risolti da una macchina astratta M usando $O(f(n))$ della risorsa R , con n dimensione dell'input

19.1 P

P è una classe di complessità che rappresenta l'insieme di tutti i problemi di decisione che **possono essere risolti in tempo polinomiale**, ovvero quei problemi tali che, dato un input, riescono a dare un output *si* o *no* in tempo polinomiale.

I problemi P sono i problemi che *possono essere risolti da una macchina di Turing deterministica in tempo polinomiale*.

19.2 NP

NP è una classe di complessità che rappresenta l'insieme di tutti i problemi di decisione per i quali **le istanze che danno *si* come risposta possono essere verificate in tempo polinomiale**. Ovvero: se la Sibilla Cumana mi dice che, dato un input x , la risposta è *si*, posso verificare la correttezza dell'affermazione in tempo polinomiale.

I problemi NP sono i problemi che *possono essere risolti da una macchina di Turing non-deterministica in tempo polinomiale*.

19.3 NP-completo

NP-completo è una classe di complessità che rappresenta l'insieme di tutti i problemi $x \in \text{NP}$ tali che **è possibile una riduzione da un qualsiasi altro problema $y \in \text{NP}$ a x in tempo polinomiale**.

Intuitivamente, possiamo risolvere y velocemente se sappiamo risolvere x velocemente.

19.4 NP-hard

Un problema x è **NP-hard** se e solo se esiste un problema y **NP-completo** tale che y è **riducibile a x in tempo polinomiale**.

Intuitivamente, questi sono i problemi che sono *almeno* difficili quanto i problemi NP-completi. I problemi NP-hard non necessariamente sono in NP e non necessariamente sono problemi di decisione. I problemi NP-hard e NP sono i problemi NP-completi. Non tutti i problemi NP-hard sono NP-completi: affinché un problema NP-hard x sia NP-completo, x deve essere in NP.

È importante notare che poiché un qualsiasi problema NP-completo può essere ridotto a qualsiasi altro problema NP-completo in tempo polinomiale e tutti i problemi in NP sono riducibili in tempo polinomiale a problemi NP-completi, **tutti i problemi NP possono essere ridotti a un qualsiasi problema NP-hard in tempo polinomiale**.

La conseguenza di queste osservazioni è che **se esiste una soluzione in tempo polinomiale a un qualsiasi problema NP-hard, allora esiste una soluzione a tutti i problemi NP in tempo polinomiale!**

